```
% ==========================================================================
%  Robot_Learning_Code.m
%  DMP + GMM/GMR Learning from Demonstration — 4D (x, y, force, time)
%  Revised: segment-wise resampling with duplicate time removal
% ==========================================================================

clearvars; clc; close all;

% ─── Dock all figures ──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────
set(0, 'DefaultFigureWindowStyle', 'docked');

% ==========================================================================
%  SECTION 1: Parameters  ← EDIT ONLY HERE
% ==========================================================================

CHAR_ID       = 'A_Uppercase';   % e.g. 'B_Uppercase', 'E_Uppercase'
MAT_FOLDER    = 'Data_for_RL_4D';
OUTPUT_FOLDER = 'RL_Character_Data_csv';

% --- Timing grid (ROS output) ---
DT_GRID = 0.010;        % s — uniform output timestep (10 ms = 100 Hz)

% --- Discontinuity detection ---
gap_thresh_mm = 3.0;    % mm — jump that signals a pen lift

% --- Dataset ---
nbData  = 200;           % samples per demo (must match Prepare_4D_mat_file)
nbDemos = 10;             % demos to use for training

% --- DMP ---
nStates     = 9;
nVar        = 5;         % [canonical s,  x,  y,  force,  t_norm]
nVarPos     = nVar - 1;  % 4 spatial DOF
beta        = 50;
alpha       = (2*beta)^0.3;
decayFactor = 1.1;
dt          = 0.01;
L           = [eye(nVarPos)*beta, eye(nVarPos)*alpha];   % 4×8

% --- GMM ---
K_gmm = 20;

% --- GMR output resolution ---
GMR_PTS_PER_SEG = 500;

% ==========================================================================
%  Derived names
% ==========================================================================
```

```
MAT_FILE = [CHAR_ID '_4D.mat'];
CHAR_KEY = CHAR_ID;

% ==========================================================================
%  SECTION 2: Load Data & Filter Demos with Consistent Segment Count
% ==========================================================================

matPath = fullfile(MAT_FOLDER, MAT_FILE);
if ~isfile(matPath)
    error('Cannot find %s\nCheck MAT_FOLDER and CHAR_ID.', matPath);
end
load(matPath, 'demos', 'avg_duration');

fprintf('Loaded %s | avg_duration = %.4f s\n', MAT_FILE, avg_duration);
```

```
Loaded A_Uppercase_4D.mat | avg_duration = 3.2391 s
```

```
% --- Segment detection for ALL demos ---
segBoundsAll = detectSegments(demos, numel(demos), gap_thresh_mm);
nSegsPerDemo = cellfun(@(s) numel(s), segBoundsAll);
modeSeg = mode(nSegsPerDemo);
fprintf('Mode segment count across all demos: %d\n', modeSeg);
```

```
Mode segment count across all demos: 2
```

```
% Keep only demos with this segment count
keep = nSegsPerDemo == modeSeg;
demos_in = demos(keep);
nTotalValid = numel(demos_in);
fprintf('Kept %d demos (out of %d) with %d segments.\n', nTotalValid,
numel(demos), modeSeg);
```

```
Kept 21 demos (out of 21) with 2 segments.
```

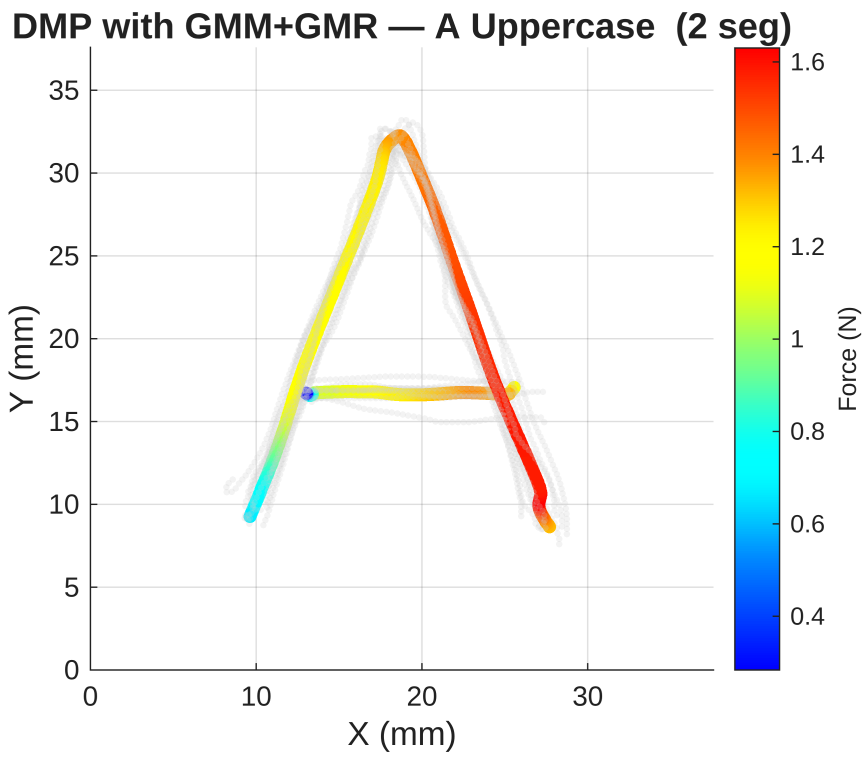
```
if nTotalValid < nbDemos
    fprintf('  Requested %d demos, but only %d available. Reducing
nbDemos.\n', nbDemos, nTotalValid);
    nbDemos = nTotalValid;
end
demos_in = demos_in(1:nbDemos);
segBounds = detectSegments(demos_in, nbDemos, gap_thresh_mm);
nSeg = modeSeg;
fprintf('Using %d segment(s) per demo (all %d demos).\n', nSeg, nbDemos);
```

```
Using 2 segment(s) per demo (all 10 demos).
```

```
% ==========================================================================
%  SECTION 3: Canonical System
% ==========================================================================
```

```
xIn    = zeros(1, nbData);
xIn(1) = 1;
for t = 2:nbData
    xIn(t) = xIn(t-1) - decayFactor * xIn(t-1) * dt;
end

% ==========================================================================
%  SECTION 4: DMP with GMM + GMR — segment-by-segment (4D)
% ==========================================================================

trajGMR_segs = cell(1, nSeg);
avgSegLens   = zeros(1, nSeg);

for seg = 1:nSeg
    segLengths = arrayfun(@(n) ...
        segBounds{n}{seg}(2) - segBounds{n}{seg}(1) + 1, 1:nbDemos);
    avgSegLen       = round(mean(segLengths));
    avgSegLens(seg) = avgSegLen;

    DataIn_seg = (1:avgSegLen) * dt;
    traj_gmm   = [];

    for n = 1:nbDemos
        s_s = segBounds{n}{seg}(1);
        s_e = segBounds{n}{seg}(2);
        pos_rs   = resampleRows(demos_in{n}.pos(:, s_s:s_e), avgSegLen);
        traj_gmm = [traj_gmm, [DataIn_seg; pos_rs]]; %#ok<AGROW>
    end

    Data_GMM_seg = traj_gmm';
    gmmodel_seg = MixtureGaussians(Data_GMM_seg, K_gmm);
    gmmodel_seg = gmmodel_seg.gmmFit(Data_GMM_seg, 100, 1e-8, false);
    gmmodel_seg = gmmodel_seg.defineQueryDim([1, 0, 0, 0, 0]);

    Querys_seg  = linspace(min(Data_GMM_seg(:,1)), max(Data_GMM_seg(:,1)),
GMR_PTS_PER_SEG)';
    gmmodel_seg = gmmodel_seg.gaussianMixtureRegression(Querys_seg);
    trajGMR_segs{seg} = gmmodel_seg.regressedTraj(:, 2:5)';  % 4 ×
GMR_PTS_PER_SEG
end

% ==========================================================================
%  SECTION 5: Convert t_norm → absolute time (per segment)
% ==========================================================================

for seg = 1:nSeg
    t_norm_seg = trajGMR_segs{seg}(4, :);
    % Force monotonicity
    for k = 2:numel(t_norm_seg)
```

```
        if t_norm_seg(k) < t_norm_seg(k-1)
            t_norm_seg(k) = t_norm_seg(k-1);
        end
    end
    trajGMR_segs{seg}(4, :) = t_norm_seg;
end
t_abs_segs = cellfun(@(s) s(4,:) * avg_duration, trajGMR_segs,
'UniformOutput', false);

% ==========================================================================
%  FIGURE 1: GMR 3D plot (unchanged, works well)
% ==========================================================================

cmap = blueRedCmap(256);
figure('Name', sprintf('DMP with GMM+GMR 4D — %s', strrep(CHAR_KEY,'_','
')), 'Color','w');
ax_gmr = axes;
for n = 1:nbDemos
    t_demo = demos_in{n}.pos(4,:) * avg_duration;
    scatter3(ax_gmr, demos_in{n}.pos(1,:), demos_in{n}.pos(2,:), t_demo, ...
             5, [0.82 0.82 0.82],
'filled','MarkerEdgeColor','none','MarkerFaceAlpha',0.25);
    hold(ax_gmr, 'on');
end
allF_gmr = cellfun(@(s) s(3,:), trajGMR_segs, 'UniformOutput', false);
fmin_gmr = min([allF_gmr{:}]); fmax_gmr = max([allF_gmr{:}]);
for seg = 1:nSeg
    t_real_seg = t_abs_segs{seg};
    cols = forceColours(trajGMR_segs{seg}(3,:), cmap, fmin_gmr, fmax_gmr);
    scatter3(ax_gmr, trajGMR_segs{seg}(1,:), trajGMR_segs{seg}(2,:),
t_real_seg, ...
             22, cols, 'filled','MarkerEdgeColor','none');
end
xlabel(ax_gmr, 'X (mm)', 'FontSize',12); ylabel(ax_gmr, 'Y (mm)',
'FontSize',12);
zlabel(ax_gmr, 'Time (s)', 'FontSize',12);
title(ax_gmr, sprintf('DMP with GMM+GMR — %s  (%d seg)',
strrep(CHAR_KEY,'_',' '), nSeg), ...
      'FontSize',13,'FontWeight','bold');
colormap(ax_gmr, cmap); clim(ax_gmr, [fmin_gmr, fmax_gmr]);
cb = colorbar(ax_gmr); cb.Label.String = 'Force (N)';
xlim(ax_gmr, [0, 37.59]); ylim(ax_gmr, [0, 37.59]);
ax_gmr.DataAspectRatioMode='auto'; ax_gmr.PlotBoxAspectRatio=[1 1 1];
ax_gmr.PlotBoxAspectRatioMode='manual';
ax_gmr.XDir='normal'; ax_gmr.YDir='normal'; ax_gmr.ZDir='reverse';
ax_gmr.XGrid='on'; ax_gmr.YGrid='on'; ax_gmr.ZGrid='on';
view(ax_gmr, 0, 90); hold(ax_gmr, 'off');
```




```
% ========================================================================
%  SECTION 6: Build final trajectory by segment-wise resampling
%  (with duplicate time removal)
% ========================================================================

t_start_offset = 0;
t_grid_all = [];
x_grid_all = [];
y_grid_all = [];
f_grid_all = [];
contact_str_all = {};

contactWindows = zeros(nSeg, 2);

for seg = 1:nSeg
    % Get segment data
    t_abs_seg = t_abs_segs{seg};
    x_seg = trajGMR_segs{seg}(1, :);
    y_seg = trajGMR_segs{seg}(2, :);
    f_seg = trajGMR_segs{seg}(3, :);

    % --- Remove duplicate timestamps (keep first) ---
    [t_abs_seg, uniqIdx] = unique(t_abs_seg, 'stable');
    x_seg = x_seg(uniqIdx);
    y_seg = y_seg(uniqIdx);
    f_seg = f_seg(uniqIdx);
```

```
    % If after removal we have fewer than 2 points, duplicate the last point
    if length(t_abs_seg) < 2
        t_abs_seg = [t_abs_seg; t_abs_seg(end)+0.001];
        x_seg = [x_seg; x_seg(end)];
        y_seg = [y_seg; y_seg(end)];
        f_seg = [f_seg; 0];
    end

    % Record segment time window (absolute times)
    t_seg_start = t_abs_seg(1);
    t_seg_end   = t_abs_seg(end);
    contactWindows(seg, :) = [t_seg_start, t_seg_end];

    % Determine grid points that lie inside this segment
    t_grid_start = ceil(t_seg_start / DT_GRID) * DT_GRID;
    t_grid_end   = floor(t_seg_end / DT_GRID) * DT_GRID;
    if t_grid_end < t_grid_start
        % Segment is shorter than one grid step: keep at least one point
        t_grid_seg = t_seg_start;
    else
        t_grid_seg = (t_grid_start : DT_GRID : t_grid_end)';
    end

    % Interpolate using linear (no overshoot)
    x_interp = interp1(t_abs_seg, x_seg, t_grid_seg, 'linear');
    y_interp = interp1(t_abs_seg, y_seg, t_grid_seg, 'linear');
    f_interp = interp1(t_abs_seg, f_seg, t_grid_seg, 'linear');
    f_interp = max(f_interp, 0);

    % Append segment points
    t_grid_all = [t_grid_all; t_grid_seg];
    x_grid_all = [x_grid_all; x_interp];
    y_grid_all = [y_grid_all; y_interp];
    f_grid_all = [f_grid_all; f_interp];
    contact_str_all = [contact_str_all; repmat({'Yes'}, size(t_grid_seg))];

    % --- Pen-lift to next segment (if any) ---
    if seg < nSeg
        t_next_abs = t_abs_segs{seg+1};
        x_next = trajGMR_segs{seg+1}(1, :);
        y_next = trajGMR_segs{seg+1}(2, :);
        f_next = trajGMR_segs{seg+1}(3, :);

        % Remove duplicates from next segment's times
        [t_next_abs, uniqNext] = unique(t_next_abs, 'stable');
        x_next = x_next(uniqNext);
        y_next = y_next(uniqNext);
        f_next = f_next(uniqNext);

        t_lift_start = t_abs_seg(end);
```

```
        t_lift_end   = t_next_abs(1);
        if t_lift_end <= t_lift_start
            t_lift_end = t_lift_start + DT_GRID;
        end

        % Generate lift grid points
        t_lift = (t_lift_start : DT_GRID : t_lift_end)';
        % Avoid duplicate of the start point (already present)
        if length(t_lift) > 1 && abs(t_lift(1) - t_lift_start) < 1e-8
            t_lift = t_lift(2:end);
        end
        if isempty(t_lift)
            t_lift = t_lift_end;
        end

        n_lift = length(t_lift);
        if n_lift == 1
            x_lift = (x_seg(end) + x_next(1))/2;
            y_lift = (y_seg(end) + y_next(1))/2;
        else
            frac = (t_lift - t_lift_start) / (t_lift_end - t_lift_start);
            x_lift = x_seg(end) + frac*(x_next(1) - x_seg(end));
            y_lift = y_seg(end) + frac*(y_next(1) - y_seg(end));
        end
        f_lift = zeros(size(t_lift));

        % Append lift points
        t_grid_all = [t_grid_all; t_lift];
        x_grid_all = [x_grid_all; x_lift];
        y_grid_all = [y_grid_all; y_lift];
        f_grid_all = [f_grid_all; f_lift];
        contact_str_all = [contact_str_all; repmat({'No'}, size(t_lift))];
    end
end

% Ensure time starts at 0
t_grid_all = t_grid_all - t_grid_all(1);
if t_grid_all(1) > 0
    t_grid_all = [0; t_grid_all];
    x_grid_all = [x_grid_all(1); x_grid_all];
    y_grid_all = [y_grid_all(1); y_grid_all];
    f_grid_all = [0; f_grid_all];
    contact_str_all = [{'No'}; contact_str_all];
end

% Sort by time (should already be sorted, but just in case)
[t_grid_all, idx] = sort(t_grid_all);
x_grid_all = x_grid_all(idx);
y_grid_all = y_grid_all(idx);
f_grid_all = f_grid_all(idx);
```

```
contact_str_all = contact_str_all(idx);

% Remove any remaining duplicates (though we already handled inside segments)
[t_grid_all, uIdx] = unique(t_grid_all, 'stable');
x_grid_all = x_grid_all(uIdx);
y_grid_all = y_grid_all(uIdx);
f_grid_all = f_grid_all(uIdx);
contact_str_all = contact_str_all(uIdx);

fprintf('Final trajectory: %d points | t = [%.3f, %.3f] s\n', ...
        numel(t_grid_all), t_grid_all(1), t_grid_all(end));
```

```
Final trajectory: 306 points | t = [0.000, 3.060] s
```

```
fprintf('  Contact: %d | Pen lift: %d\n', ...
        sum(strcmp(contact_str_all, 'Yes')), sum(strcmp(contact_str_all,
'No')));
```

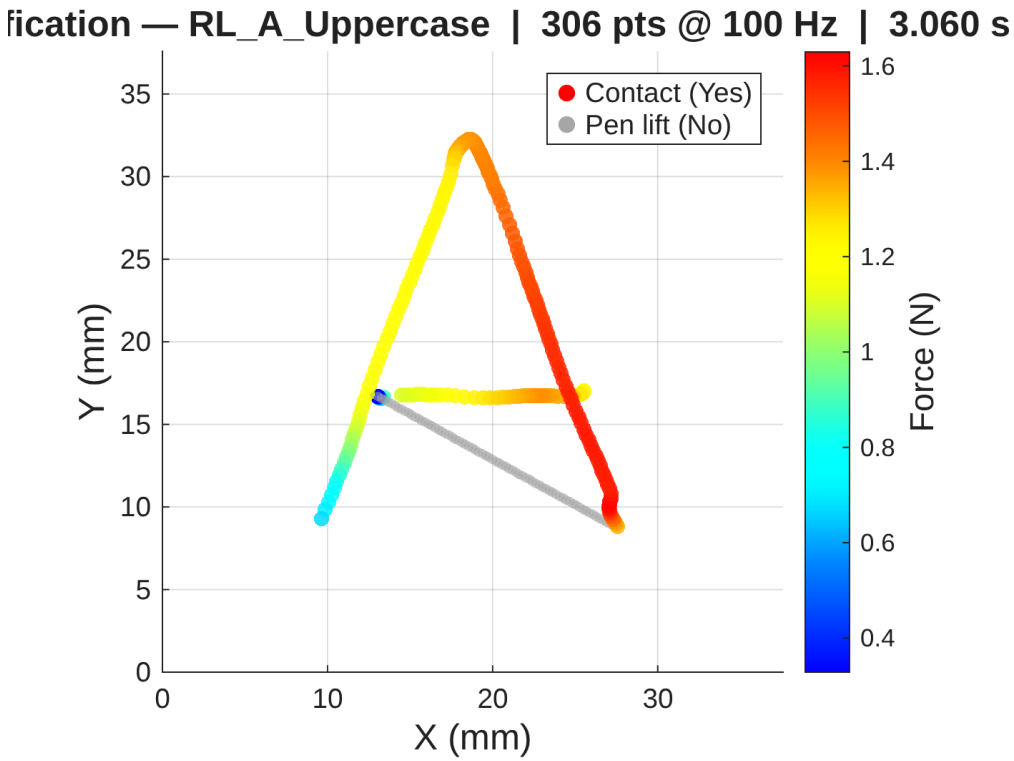
```
  Contact: 234 | Pen lift: 72
```

```

% ==========================================================================
%  SECTION 7: Write CSV
% ==========================================================================

if ~isfolder(OUTPUT_FOLDER)
    mkdir(OUTPUT_FOLDER);
end
outFile = fullfile(OUTPUT_FOLDER, sprintf('RL_%s.csv', CHAR_KEY));

fid = fopen(outFile, 'w');
if fid == -1
    error('Cannot open: %s', outFile);
end
fprintf(fid, 'Character,x (mm),y (mm),Force (N),Time (s),Contact (Yes/No)
\n');
for k = 1:numel(t_grid_all)
    fprintf(fid, '%s,%.4f,%.4f,%.4f,%.3f,%s\n', ...
            CHAR_KEY, x_grid_all(k), y_grid_all(k), f_grid_all(k),
t_grid_all(k), contact_str_all{k});
end
fclose(fid);
fprintf('Saved: %s\n', outFile);
```

```
Saved: RL_Character_Data_csv/RL_A_Uppercase.csv
```

```

% ==========================================================================
%  SECTION 8: Verification 3D plot
% ==========================================================================

isC = strcmp(contact_str_all, 'Yes');
```

```
f_c = f_grid_all(isC);
fmin_v = max(min(f_c), 0); fmax_v = max(f_c);

fNorm = min(max((f_c-fmin_v)./(fmax_v-fmin_v+eps), 0), 1);
cIdx = max(1, round(fNorm*255)+1);
cols_c = cmap(cIdx, :);

fig_v = figure('Name', sprintf('Verification 3D — RL_%s', CHAR_KEY),
'Color','w');
ax_v = axes('Parent', fig_v);
scatter3(ax_v, x_grid_all(isC), y_grid_all(isC), t_grid_all(isC), ...
         30, cols_c,
'filled','MarkerEdgeColor','none','MarkerFaceAlpha',0.85);
hold(ax_v, 'on');
if any(~isC)
    scatter3(ax_v, x_grid_all(~isC), y_grid_all(~isC), t_grid_all(~isC), ...
             12, [0.65 0.65 0.65],
'filled','MarkerEdgeColor','none','MarkerFaceAlpha',0.55);
end
xlabel(ax_v,'X (mm)','FontSize',13); ylabel(ax_v,'Y (mm)','FontSize',13);
zlabel(ax_v,'Time (s)','FontSize',13);
title(ax_v, sprintf('Verification — RL\\_%s  |  %d pts @ %.0f Hz  |  %.3f
s', ...
      strrep(CHAR_KEY,'_','\_'), numel(t_grid_all), 1/DT_GRID,
t_grid_all(end)), ...
      'FontSize',13,'FontWeight','bold');
colormap(ax_v, cmap); clim(ax_v, [fmin_v, fmax_v]);
cb_v = colorbar(ax_v); cb_v.Label.String='Force (N)'; cb_v.Label.FontSize=12;
xlim(ax_v, [0,37.59]); ylim(ax_v, [0,37.59]);
ax_v.DataAspectRatioMode='auto'; ax_v.PlotBoxAspectRatio=[1 1 1];
ax_v.PlotBoxAspectRatioMode='manual';
ax_v.XDir='normal'; ax_v.YDir='normal'; ax_v.ZDir='reverse';
ax_v.XGrid='on'; ax_v.YGrid='on'; ax_v.ZGrid='on';
view(ax_v, 0, 90);
h_c = scatter3(ax_v, NaN, NaN, NaN, 30, [1 0 0], 'filled');
h_l = scatter3(ax_v, NaN, NaN, NaN, 12, [0.65 0.65 0.65], 'filled');
legend(ax_v, [h_c, h_l], {'Contact (Yes)', 'Pen lift (No)'}, 'Location',
'northeast', 'FontSize', 10);
hold(ax_v, 'off');
```




```
% ==========================================================================
%  LOCAL FUNCTIONS (unchanged)
% ==========================================================================

function segBounds = detectSegments(demos_in, nbDemos, gap_thresh_mm)
    segBounds = cell(1, nbDemos);
    for n = 1:nbDemos
        xy    = demos_in{n}.pos(1:2,:);
        diffs = sqrt(sum(diff(xy,1,2).^2,1));
        gapAt = find(diffs > gap_thresh_mm);
        bounds = {}; prev = 1;
        for gi = 1:numel(gapAt)
            bounds{end+1} = [prev, gapAt(gi)]; %#ok<AGROW>
            prev = gapAt(gi)+1;
        end
        bounds{end+1} = [prev, size(xy,2)];
        segBounds{n}  = bounds;
    end
end

function out = resampleRows(mat, newLen)
    [D, N] = size(mat);
    if N == newLen; out = mat; return; end
    tOld = linspace(0,1,N); tNew = linspace(0,1,newLen);
    out  = zeros(D, newLen);
    for d = 1:D; out(d,:) = spline(tOld, mat(d,:), tNew); end
end
```

```
function cols = forceColours(fVals, cmap, fmin, fmax)
    if nargin < 3; fmin=min(fVals); fmax=max(fVals); end
    fNorm = min(max((fVals-fmin)./(fmax-fmin+eps),0),1);
    cIdx  = max(1, round(fNorm*(size(cmap,1)-1))+1);
    cols  = cmap(cIdx,:);
end

function cmap = blueRedCmap(n)
    if nargin < 1; n = 256; end
    keys = [0 0 1; 0 0.5 1; 0 1 1; 0.5 1 0.5; 1 1 0; 1 0.5 0; 1 0 0];
    xi   = linspace(0,1,size(keys,1)); xq = linspace(0,1,n);
    cmap = min(max(interp1(xi,keys,xq,'pchip'),0),1);
end
```